```
// clear "Reg(m)"
affine.for %i = 0 to 2 {
  affine.store -inf, %m[%i] : memref<2xf32>
}
// reduce "Reg(m)" by "For" loop
affine.for %i = 0 to 2 {
  affine.for %j = 0 to 4 {
    %0 = affine.load %m[%i] : memref<2xf32>
    %1 = affine.load %acc_s[%i, %j] : memref<2x4xf32>
    %2 = arith.maxf %0, %1 : f32
    affine.store %2, %m[%i] : memref<2xf32>
  }
}
// reduce "Reg(m)" by "__shfl_xor_sync"
affine.for %i = 0 to 2 {
  %0 = affine.load %m[%i] : memref<2xf32>
  %shuffleResult, %valid = gpu.shuffle xor %0, 4, 8 : f32  // __shfl_xor_sync(0xffffffff, %0, 4, 8);
  %1 = arith.maxf %0, %shuffleResult : f32
  affine.store %1, %m[%i] : memref<2xf32>

  %0 = affine.load %m[%i] : memref<2xf32>
  %shuffleResult, %valid = gpu.shuffle xor %0, 2, 8 : f32  // __shfl_xor_sync(0xffffffff, %0, 2, 8);
  %1 = arith.maxf %0, %shuffleResult : f32
  affine.store %1, %m[%i] : memref<2xf32>

  %0 = affine.load %m[%i] : memref<2xf32>
  %shuffleResult, %valid = gpu.shuffle xor %0, 1, 8 : f32  // __shfl_xor_sync(0xffffffff, %0, 1, 8);
  %1 = arith.maxf %0, %shuffleResult : f32
  affine.store %1, %m[%i] : memref<2xf32>
}
```